

OSSP (Operating Systems And Systems Programming) - 25CS2104E

Roll No. : 2520030027

Name: Venkata Sai Krishna

SEC-8

SKILL-3:

Aim: To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality.

To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind.

PROCEDURE :

1. Command History Management Using Escape Sequences :

```
venkatsaikrishna5@VSK-HP:~$ mkdir skill3
venkatsaikrishna5@VSK-HP:~$ cd skill3
venkatsaikrishna5@VSK-HP:~/skill3$ nano history.c
venkatsaikrishna5@VSK-HP:~/skill3$ gcc history.c -o history
venkatsaikrishna5@VSK-HP:~/skill3$ ./history
Simple Command History
Type 'history' to display commands
Type 'prev' for previous command
Type 'next' for next command
Type 'exit' to quit

shell> ls
Command stored in history.
shell> pwd
Command stored in history.
shell> date
Command stored in history.
shell> history
1  ls
2  pwd
3  date
shell> prev
Previous command: date
shell> next
Next command: No newer command.
shell> exit
History memory released.
```

2. Dynamic Memory Management and Memory Leak Detection Using :

```
venkatsaikrishna5@VSK-HP:~/skill3$ nano memory.c
venkatsaikrishna5@VSK-HP:~/skill3$ gcc memory.c -o memory
venkatsaikrishna5@VSK-HP:~/skill3$ ./memory
Enter commands (type 'done' to finish):
Enter command: ls
Enter command: pwd
Enter command: date
Enter command: whoami
Enter command: dcc
Enter command: done

Stored commands:
1. ls
2. pwd
3. date
4. whoami
5. dcc

All allocated memory released successfully.
```

3. Dynamic Memory Management and Memory Leak Detection Using Valgrind

```
venkatsaikrishna5@VSK-HP:~/skill3$ gcc -g memory.c -o memory
venkatsaikrishna5@VSK-HP:~/skill3$ valgrind --leak-check=full .
./memory
==958== Memcheck, a memory error detector
==958== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==958== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==958== Command: ./memory
==958==
Enter commands (type 'done' to finish):
Enter command: ls
Enter command: pwd
Enter command: date
Enter command: done

Stored commands:
1. ls
2. pwd
3. date

All allocated memory released successfully.
==958==
==958== HEAP SUMMARY:
==958==   in use at exit: 0 bytes in 0 blocks
==958==   total heap usage: 7 allocs, 7 frees, 2,108 bytes allocated
==958==
==958== All heap blocks were freed -- no leaks are possible
==958==
==958== For lists of detected and suppressed errors, rerun with: -s
==958== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Code:

```
GNU nano 8.7.1          history.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_HISTORY 10
int main()
{
    char *history[MAX_HISTORY];
    int count = 0;
    char input[100];
    printf("Simple Command History\n");
    printf("Type 'history' to display commands\n");
    printf("Type 'prev' for previous command\n");
    printf("Type 'next' for next command\n");
    printf("Type 'exit' to quit\n\n");
    while (1)
    {
        printf("shell> ");
        fgets(input, sizeof(input), stdin);
        input[strcspn(input, "\n")] = '\0';
        if (strcmp(input, "exit") == 0)
            break;
        if (strcmp(input, "history") == 0)
        {
            for (int i = 0; i < count; i++)
                printf("%d %s\n", i + 1, history[i]);
            continue;
        }
        if (strcmp(input, "prev") == 0)
        {
            if (count > 0)
                printf("Previous command: %s\n", history[count-1]);
            else
                printf("No previous command.\n");
            continue;
        }
    }
}
```

```
    if (strcmp(input, "next") == 0)
    {
        printf("Next command: No newer command.\n");
        continue;
    }

    if (strlen(input) > 0)
    {
        if (count < MAX_HISTORY)
        {
            history[count] = malloc(strlen(input) + 1);
            strcpy(history[count], input);
            count++;
        }

        printf("Command stored in history.\n");
    }
}

for (int i = 0; i < count; i++)
    free(history[i]);

printf("History memory released.\n");

return 0;
}
```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Node
{
    char *data;
    struct Node *next;
};

int main()
{
    int size = 2;
    int count = 0;
    char **commands = malloc(size * sizeof(char *));
    if (commands == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }
    char input[100];
    printf("Enter commands (type 'done' to finish):\n");
    while (1)
    {
        printf("Enter command: ");
        fgets(input, sizeof(input), stdin);
        input[strcspn(input, "\n")] = '\0';
        if (strcmp(input, "done") == 0)
            break;
        if (count == size)
        {
            size *= 2;
            char **temp = realloc(commands,
                                   size * sizeof(char *));
            if (temp == NULL)
            {
                printf("Resize failed.\n");
            }
        }
    }
}
```

```

        printf("Resize failed.\n");
        break;
    }
    commands = temp;
}
commands[count] = malloc(strlen(input) + 1);
if (commands[count] == NULL)
{
    printf("Memory allocation failed.\n");
    break;
}
strcpy(commands[count], input);
count++;
}
printf("\nStored commands:\n");
for (int i = 0; i < count; i++)
    printf("%d. %s\n", i + 1, commands[i]);
for (int i = 0; i < count; i++)
    free(commands[i]);
free(commands);
printf("\nAll allocated memory released successfully.\n");
return 0;
}

```

Observation :

1. Commands were stored successfully in the history buffer.
2. Previous and next command navigation was tested.
3. Dynamic memory was allocated using malloc().
4. The memory array was resized using realloc().
5. Input size was controlled to avoid buffer overflow.
6. Allocated memory was released using free().
7. Valgrind confirmed that there were no memory leaks.

Result :

The experiments successfully implemented **command history management and dynamic memory management**. Previous and next command recall worked correctly, memory buffers were dynamically allocated and resized, and Valgrind verified that the allocated memory was properly released without memory leaks.